

Self-Evolving Auto Research 系统

从执行层优化到跨任务经验复用

周浩洋 · Jilin University, School of Software

Research Assistant · 2025.04 — Present

目录

01

创新点1·洞察：5层模型 与瓶颈

Auto Research 分层·执行层
bottleneck

02

创新点2·Infra： AgentGenesis 框架

并行探索·适配器模式·自然语言转
环境

03

创新点3·算法：History Graph

跨任务经验复用·图结构共享记忆

04

实验与 Future Work

72 数据集阈值·对比实验设计

创新点1·洞察

Auto Research 5层分层模型与当前 bottleneck

创新点1 · Auto Research 的两种理解

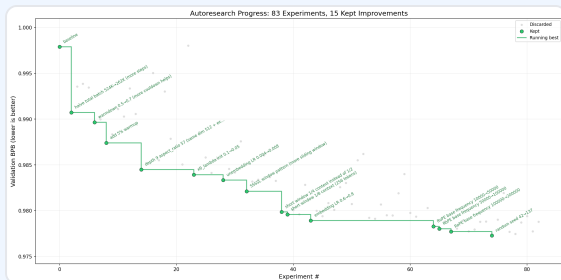
范式 A : 执行层优化

用 Coding Agent 对可量化评分任务进行调优迭代

典型：ML 超参数搜索、RL 经典小游戏 (Atari)

代表：Karpathy / autoresearch, 翁家翌 HL (启发式学习)

核心：实验 → 反馈 → 迭代 (第 4 层)



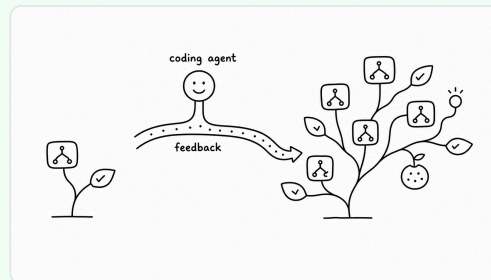
范式 B : 端到端论文生成

全自动或 Human-in-the-Loop 完成论文全流程

搜集文献 → 发现 Idea → 设计验证 → 执行实验 → 撰写论文

代表：AI-Scientist-v2, Auto-claude-code-research-in-sleep

核心：第 1-5 层端到端闭环



提升前者有助于后者整体提升，后者是对前者的延伸，两者不矛盾

创新点1 · 我们提出的 Auto Research 5 层分层模型

Layer 1 · 提出或定义问题

瓶颈：Context Learning（综述性论文）

Layer 2 · 搜集学习论文

瓶颈：Context Learning（综述性论文）

Layer 3 · 提出可能的 Idea 集合

瓶颈：Context Learning（综述性论文）

Layer 4 · 设计验证实验并执行 ★ 我们的聚焦点

瓶颈：Code 出正确的实验得到正确结果。Idea 可批量生成，但快速有效验证并不容易

Layer 5 · 撰写论文与 Rebuttal

瓶颈：Context Learning（综述性论文）

对于 CS 非综述性论文，bottleneck 在第 4 层；对于综述性论文，bottleneck 在 1,2,3,5 层的 Context Learning

创新点1 · 第 4 层的核心挑战

别人的痛点

- × 单次任务内搜索，不跨任务复用 (AIDE / OpenEvolve)
- × 多 Agent 共享文件系统，无结构化知识抽象 (CORAL)
- × 需要 10^4 次迭代才能收敛 (RL)
- × 反馈信号简单，无长期知识库

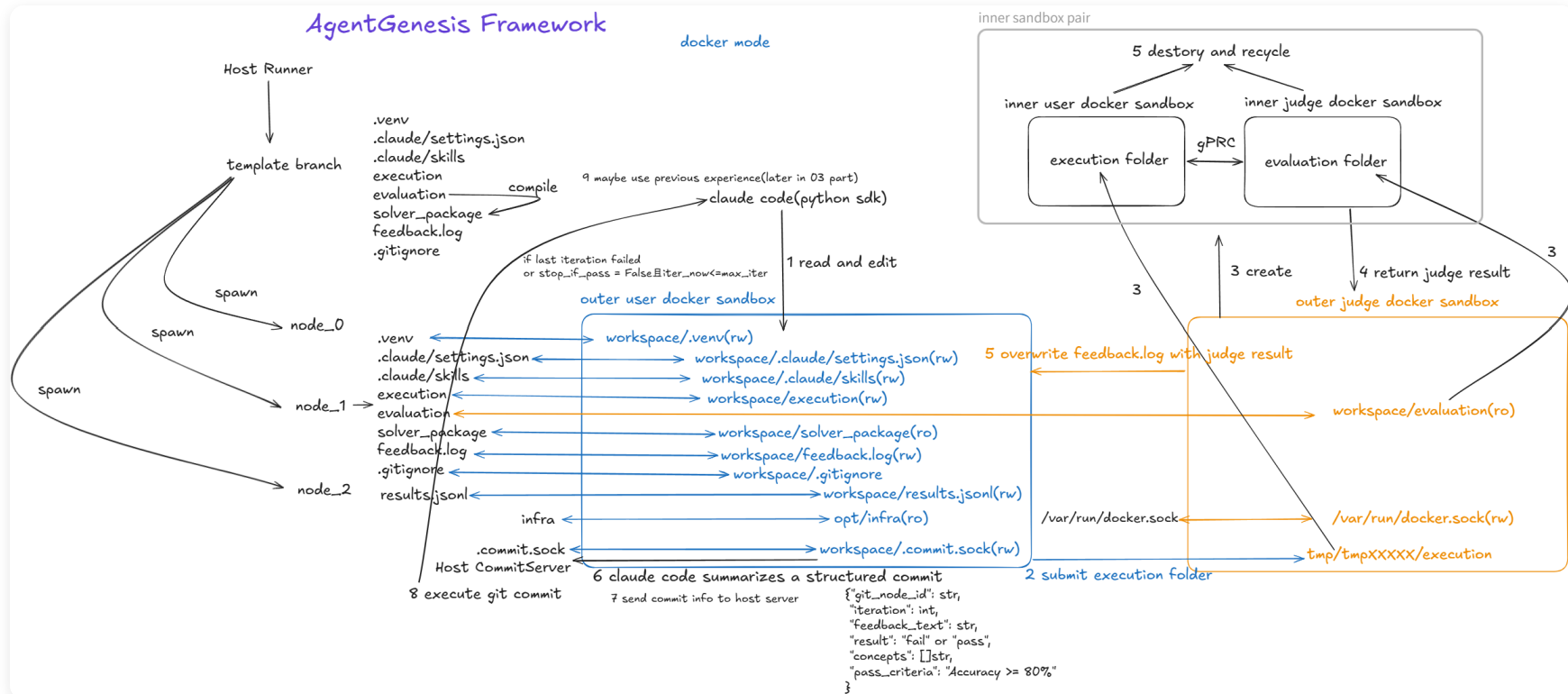
我们的方案

- ✓ 跨任务历史经验图复用 (History Graph)
- ✓ 多 Agent 之间图结构作为共享记忆
- ✓ 只需 10 次迭代 (LLM + 经验复用)
- ✓ 结构化五元组反馈 (feedback / strategy / concept / result / pass_criteria)

创新点2 · Infra

AgentGenesis 框架：把任何 Idea 转化为可迭代可评测的执行层环境

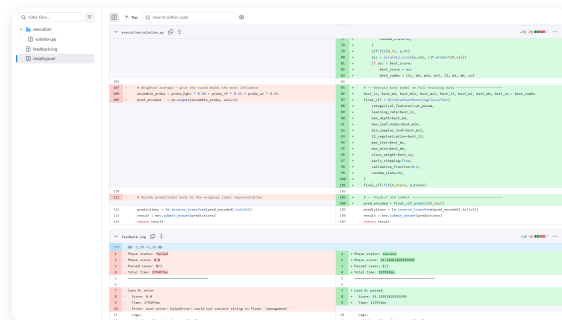
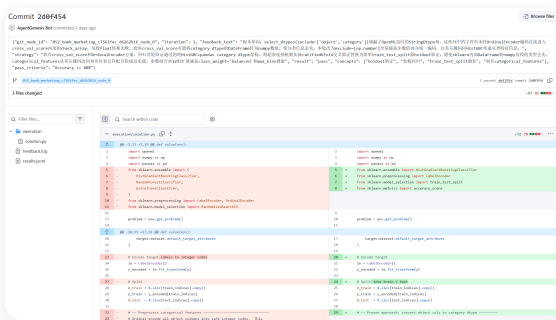
创新点2 · AgentGenesis 框架架构



创新点2 · Skill 包目录结构

```
AgentGenesis/skills/author-agentgenesis-problem/  
├─ SKILL.md # Skill manifest & entry point  
├─ examples/ # 11 official problem patterns  
├─ manuals/出题指南.md # Complete problem-authoring guide  
├─ references/ # Format, patterns, quality rubric  
│   ├── problem-format.md  
│   ├── runtime-patterns.md  
│   ├── progressive-disclosure.md  
│   └── quality-rubric.md  
├─ templates/ # Design brief & skeletons  
│   ├── problem-design-brief.md  
│   ├── single-agent-problem.md  
│   └── isolated-multi-agent-problem.md  
└─ problems/ # 11 official examples  
    ├── maze/、structured_output/、parallel_weather/  
    ├── sports_shopping/、resilient_scraper/  
    ├── tool_creator_challenge/、log_hunter/  
    └── werewolf/、microservice_avalanche/ ...
```

创新点2·迭代案例：从 Fail 到 Pass



代码 Diff：模型替换

反馈驱动优化

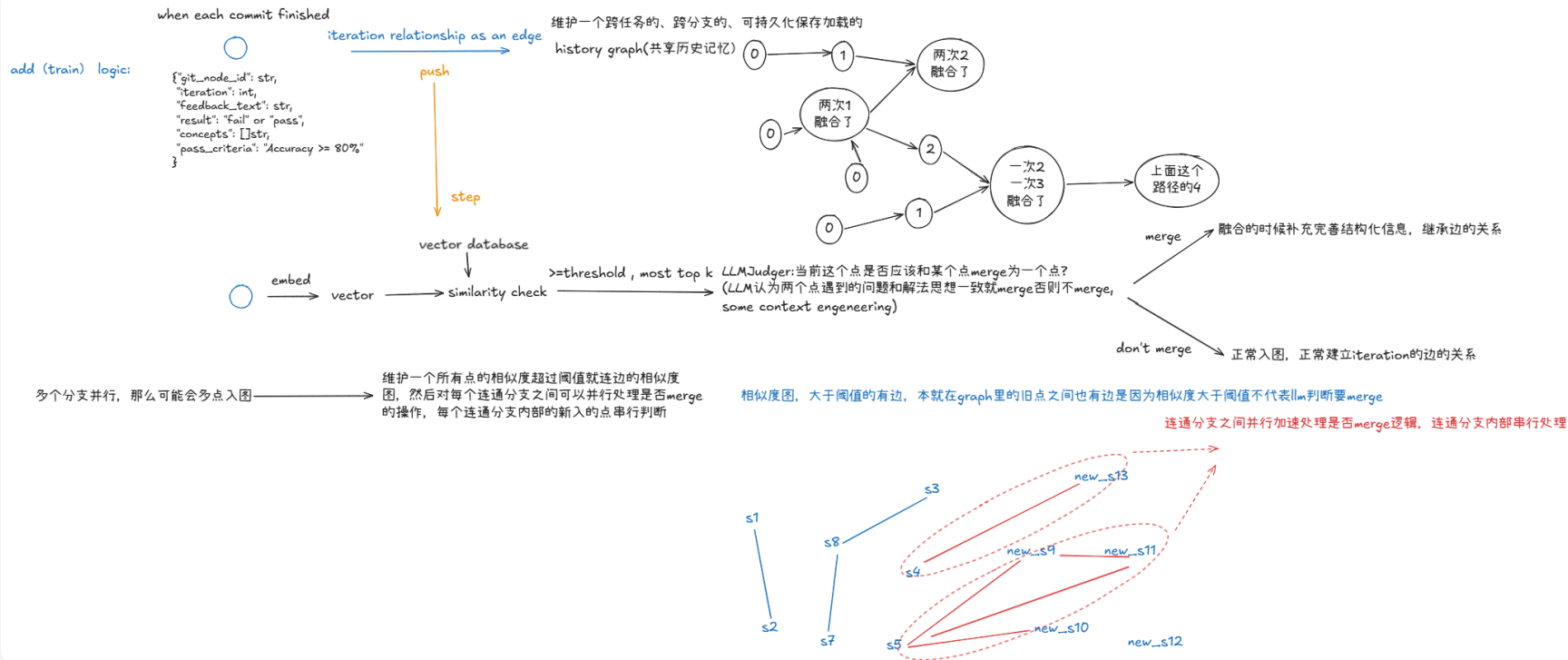
Commit 结构化记录

以 052_bank_marketing 为例：Iteration 0 失败 → 诊断 StringType 问题 → 换 OrdinalEncoder → Iteration 4 通过 (91.24%)

创新点3·算法

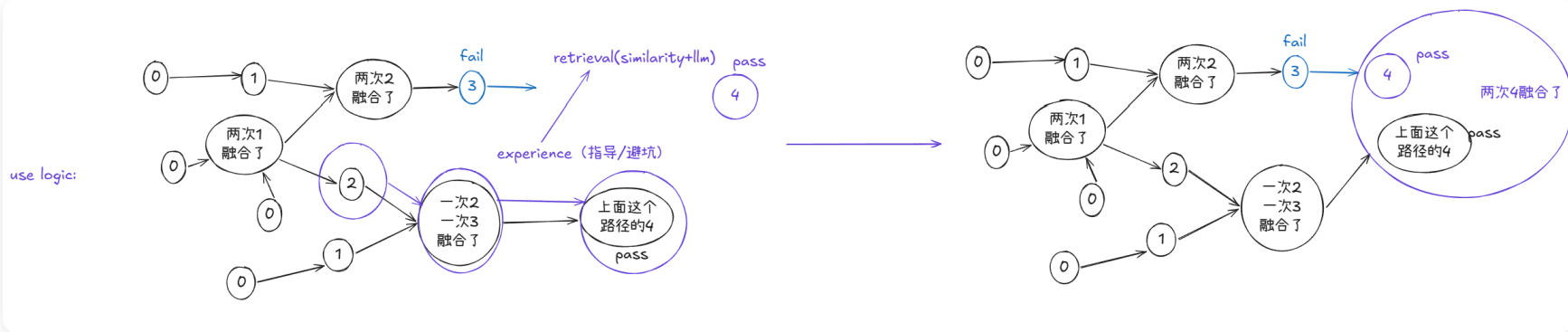
History Graph：跨任务经验复用的图结构记忆

创新点3 · Add Logic



Commit → Embedding → 相似度聚类 → LLM Merge → History Graph

创新点3 · Use Logic



当前路径 → Retrieval(similarity+LLM) → 指导/避坑经验注入 Prompt

创新点3 · 为什么 History Graph 是独特的？

记忆机制

唯一跨不同任务+跨同一任务不同分支的共享记忆

唯一多 Agent 之间图结构作为共享记忆

协作模式

其他系统只支持空间上的协作

我们支持时空上都进行协作（历史经验 + 当前并行）

反馈信号

五元组：feedback + strategy + concepts + result + pass_criteria

恰好是“检索相似问题”所需的全部信息

CORAL 共享文件系统 · AIDE/OpenEvolve 同任务内搜索 · 我们：跨任务历史经验图复用

创新点3·与现有系统的对比

维度	Ours	CORAL	AI-Scientist-v2	AIDE	OpenEvolve	GEPA	FunSearch	autoresearch
系统定位	多Agent自主编码 + 跨任务经验复用	多Agent自主编码+共享文件系统	端到端科研自动化	ML工程树搜索	进化式代码优化	反射式Prompt/代码进化	数学发现程序搜索	单GPU训练自治
架构家族	Search-Based + Multi-Agent 融合	Multi-Agent + Skill-Based	Search-Based (Tree)	Search-Based (Tree)	Search-Based (Evolutionary)	Search-Based (Evolutionary)	Search-Based (Evolutionary)	Sequential Pipeline
搜索拓扑	异步并行 + 历史图检索 + Agentic Search	异步并行 + 心跳调度	Agentic Tree Search (BFTS)	解树 (UCB/贪婪)	MAP-Elites + 岛屿迁移	Pareto前沿 + 反射突变	遗传程序搜索 (islands)	线性迭代
记忆架构	共享图结构记忆 (History Graph)	共享文件系统池 .coral/public/	树节点状态	解树节点	MAP-Elites网格 + 档案库	Pareto候选池	程序档案库	Git历史
知识复用范围	跨任务、跨会话、可累积	同任务内共享	同任务内树回溯	同任务内树回溯	同任务内岛屿迁移	同任务内 Pareto 合并	同任务内遗传继承	无
进化机制	Embedding 相似度 + LLM 合并决策	心跳触发 (reflect/consolidate/pivot)	树扩展 + Debug 回溯	Patch 生成 + 剪枝	LLM 变异 + 双选择	LLM 反射诊断 + 定向突变	随机组合 + 评估筛选	Agent 自主编辑
Agent 协作模式	并行竞争 + 图记忆共享	并行自主 + 元控制器心跳	单Agent树探索	单Agent树探索	岛屿隔离 + 定期迁移	单线程候选进化	分布式岛屿	单Agent
反馈深度	结构化五元组 (feedback/strategy/result/concept/criteria)	复合评分 + Agent notes	实验指标 + 报错信息	评估指标	标量评分 + Artifacts	执行轨迹全量反射	标量评分	标量 (val_bpb)
生命周期覆盖	s4 (Coding)	s4 (Coding)	s1-s5 (Creation→Writing)	s4 (Coding)	s4 (Coding)	s4 (Coding/Config)	s4 (Coding)	s4 (Coding)

Part 4 · 实验与 Future Work

72 数据集 Benchmark · 阈值测定 · 对比实验设计

实验 1 : pass_criteria 阈值测定 (Pilot)

Benchmark

OpenML-CC18-AG
72 个 tabular 分类任务

初始标准

Accuracy $\geq$ 0.8
stop_on_pass = False

Threshold 计算

跑 3 次
mean(best_score)

Dataset	Threshold	Min	Max
001_kr_vs_kp	99.79	99.69	100.0
003_balance_scale	100.0	100.0	100.0
014_pendigits	99.55	99.45	99.64
022_eucalyptus	68.92	63.51	77.03
069_CIFAR_10	45.41	41.5	53.23

72 个数据集全部完成 · 完整数据见 `exp/pilot/THRESHOLDS.md`

Future Work · 待验证实验

实验 2

跨任务经验复用 vs 无复用 Baseline
验证 History Graph 跨任务复用是否加速收敛

实验 3

多 Branch 共享 Graph vs CORAL vs Independent
验证结构化图记忆是否优于文件系统共享

实验 4

LLM Merge 质量验证
定量 + 定性评估 Merge 是否提炼出通用经验

实验 2：跨任务经验复用有效吗？

Baseline (无历史复用)

history_mode = disabled

每个任务从零开始探索

打乱数据集顺序，消除顺序偏差

记录：iterations_to_first_pass, success_rate@K, best_score

Ours_Full (跨任务历史复用)

history_mode = full, history_scope = cross-task

打乱数据集顺序，历史经验累积复用

记录：同上 + graph_compression (raw_commits / merged_nodes)

max_iteration = 10

验证指标：若 ours_full 的 iterations_to_first_pass ↓ 且 success_rate@K ↑ 且 best_score ↑
→ 证明 跨任务经验复用有效 (72 数据集 × 3 rounds)

实验 3：多 Branch 共享 Graph vs CORAL vs Independent

条件	描述	共享机制
Independent	3 个 branch 完全独立，各自维护空历史图，互不共享	无
CORAL	3 个 agent 同任务共享 .coral/public/ 文件系统	文件系统共享
Shared Graph (Ours)	3 个 branch 同时启动，共享同一个 merged history graph	结构化图记忆 (embedding + merge + relevance)

方法

每组 3 个并行 branch (num_nodes=3)，同时启动
每个 branch：max_iter=10，3 组重复

指标

avg_iterations_to_pass / min_iterations_to_pass
best_score_of_all_branch

预期：Shared Graph (Ours) > CORAL 文件系统共享 > Independent

实验 4 : LLM Merge 质量验证

定量分析

retrieval_hit_rate_merged : merged 节点后续被检索命中次数 / 总检索次数

retrieval_hit_rate_raw : 假设原始节点未被 merge , 模拟被检索命中次数

compression_ratio : $\text{len}(\text{raw_nodes}) / \text{len}(\text{merged_nodes})$

concept_coverage : merged 节点 concepts 并集 / 原始节点 concepts 并集

定性分析 (抽样 20 组)

Feedback 质量 : merged 是否更 general、更准确描述问题模式？

Strategy 质量 : merged 是否融合两种做法，形成更通用修复思路？

信息损失 : 是否有独特且重要的信息在 merge 后丢失？

评估：3 分制（1=差，2=一般，3=好），预期均值 > 2.0

预期结论

- **retrieval_hit_rate_merged** > **retrieval_hit_rate_raw** : merge 后的节点更 general，更容易被后续检索命中 → 证明 merge 提炼出了通用模式
- **compression_ratio** > 2 : merge 显著压缩了历史空间 → 证明不是简单拼接，而是真正聚合了经验
- 定性评分均值 > 2.0 : LLM merge 确实提炼出有意义的通用经验，而非生硬拼接或信息丢失

近期与远期计划

近期

 跑通实验 2/3/4 全量对比 (72 数据集 × 3 rounds)

 论文撰写 (执行层优化 + History Graph)

远期

 向 Layer 1-3, 5 延伸 (端到端论文生成)

 开源 AgentGenesis 框架与 Benchmark



谢谢聆听

Q & A